

Dongyeon Kim (Leo)

2026.09.18

CSSE 313 Artificial Intelligence

Lab #2 Reflection

Predictions: Do you think the network can learn XOR? Justify your prediction. If you are confident that you can train it, please give a sense of expected hyperparameter values.

I think that the network will be able to predict the XOR gate with multi-layer structure because XOR is not linearly separable. In Lab 1, I built a single-layer perceptron which was only able to solve linearly separable problems. However, the multi-layer perceptron would have a hidden layer with non-linear activation, allowing the network to carve the input space into two regions that XOR needs.

Experiment:

Improving the performance of model for XOR

What I will change:

1. I will gradually change the learning rate of the perceptron from 0.01 to 100.
2. All else held fixed to:
 - a. Structure of the perceptron: 2 inputs, 2 nodes in 1 hidden layer, 1 output.
 - b. Error threshold of 0.05
 - c. Max epoch of 5000
 - d. Sigmoid activation function across the network
 - e. Weights set to random
 - f. Seeds list for trials [1 through 10]
 - g. Bias of 0

What I predict will happen:

In Lab 1, I predicted that a lower learning rate would perform better, based on the general knowledge that low learning rates give more careful and stable convergence. For this lab, I am revising that prediction, in which I predict that a higher learning rate will perform better than a low one for training XOR on this network within the *max_epochs* of 5000

Why (mechanism I expect):

Because the backpropagation step computes its weight update by multiplying the incoming error by the derivative sigmoid function (y'), which is $y*(1-y)$. The output of the derivative sigmoid function is maxed at 0.25 when $y = 0.5$, and near zero when a neuron is saturated to be near 0 or 1. That derivative gets chained through the different layers of the network, causing the amount of change roughly ($\text{learning_rate} * (\leq 0.25)^2$). Within 5000 epochs, the model would not be able to escape much from its initial weight value, with average error sitting nearly 0.5, which means it is nearly unlearned. Raising the learning rate would compensate for the built-in shrinkage from the sigmoid derivative, which is why I expect it to perform better here than worse.

What would surprise me:

If a low learning rate ranging from $[0.01, 1]$ converged just fine given the 5000 max epoch budget. It would mean that the sigmoid-derivative shrinkage I'm predicting wasn't actually the bottleneck, but something else was.

Experiment:

Run	Learning Rate	Max Epoch	Weight	Seed	Average Epochs to Converge	Target Successful
1	0.01	5000	Random	[1-10]	> 5000.00	0/10
2	0.1	5000	Random	[1-10]	> 5000.00	0/10
3	0.5	5000	Random	[1-10]	2542.00	10/10
4	1	5000	Random	[1-10]	1259.90	10/10
5	5	5000	Random	[1-10]	330.17	6/10
6	10	5000	Random	[1-10]	295.57	7/10
7	15	5000	Random	[1-10]	586.14	7/10
8	20	5000	Random	[1-10]	568.00	7/10
9	30	5000	Random	[1-10]	1421.00	4/10
10	40	5000	Random	[1-10]	> 5000.00	0/10
11	50	5000	Random	[1-10]	> 5000.00	0/10

Test Results:

<https://drive.google.com/drive/folders/1iM97fHMqZctAO2SyWOu7Da6-9mn7Bv6m?usp=sharing>

Mechanism:

The results confirmed the core prediction that the very low learning rates (0.01, 0.1) never converged within 5000 epochs with 0/10 trials successful. This showed the consistency with the sigmoid derivative shrinkage argument, in which the effective step size was too small to escape the initial values in the epoch budget that was given. Once the learning rate reached (0.5,

1), the network converged reliably for 10/10 trials successful for both. Onwards, average epochs to converge dropped sharply as the learning rate increased to (5, 10) with $2542 \rightarrow 1260 \rightarrow \sim 300$ epochs, showing that larger steps do speed up the escape from the expected stalling point. At the high end with learning rates ranging (40, 50), the model collapsed back to 0/10, showing the speed vs stability trade off rather than my simple “higher is always better” trend.

Reconciliation:

I predicted:

I predicted that the higher learning rate would perform better than the low ones when training the XOR network within 5000 epoch budget, because the sigmoid derivative shrinks the effective step size well below the nominal learning rate. Thus, a small rate like 0.01 would leave the network near its initial error of ~ 0.5 instead of learning.

What actually happened:

The prediction was partially correct, but was incomplete. The low learning rates (0.01, 0.1) did fail exactly as predicted with 0/10 convergence, confirming the shrinkage bottleneck. But the performance did not just keep improving as the learning rate increased. It peaked in reliability at the learning rate ranging (0.5, 1) with 10/10 convergence, then reliability actually declined through (5, 30) range with 6, 7, 7, 4 / 10 convergence even while the average epoch to converge for the successful runs kept dropping, before collapsing back to 0/10 at learning rate (40, 50). Thus, the true relationship was not the higher the learning rate the better it performs, but it was an inverted U. Too low of the learning rate fails to move at all and too high of the learning rate overshoots and collapses, showing that there is a specific mid range where both speed and reliability are good.

What my mental model was missing:

I reasoned about the low end of the learning rate range about the shrinkage problem, but did not build a symmetric expectation for the high end. I implicitly treated the higher learning rate as a single lever for fixing the shrinkage problem, without accounting for the fact that a learning rate large enough would be able to jump past the minimum entirely.

Updated rule of thumb:

For a small network trained on a non linear separable problem like the XOR gate, there is a sweet spot range where it does not follow the general rule like “lower is safer” or “higher is faster” relationship. The lower bound is set by how much activation’s derivative shrinks the gradient and the upper bound is set by how large a step the loss surface can tolerate before overshooting.